System Design Document: Food Waste Reduction Platform

Workshop 2 — Systems Design (Semester 2026-I)

Luna Alejandra Sandoval Rodriguez
20241020053
Systems Analyst / Project Lead

Cristian Camilo Bonilla Lizarazo
20241020015
Systems Analyst / Frontend Developer

Juan Sebastian Bravo Rojas
20241020004
Systems Analyst / Database Specialist

Nicolas Rodriguez Granados
2024102037
Systems Analyst / Backend Developer

***Abstract*—In Bogotá, cafeterias and local restaurants generate daily surplus food suitable for consumption, yet coordination barriers and short freshness windows contribute to Colombia's significant annual food waste (approximately 9.7 million tons), exacerbating environmental damage and food insecurity. This document, as the direct continuation of Workshop 1's systems analysis, details the technical design of a mobile Food Waste Reduction Platform that enables real-time, geospatially limited redistribution within a 3 km radius around the Engineering Faculty of Universidad Distrital Francisco José de Caldas. The architecture features a React Native frontend, NestJS backend with PostGIS geospatial matching, serialized priority-based algorithm (favoring charities), staggered notifications, no-show mitigation via reliability scoring and reassignment, and automatic impact metrics. By addressing Workshop 1's 17 measurable requirements and 10 critical risks, the design achieves feasibility for 80% surplus recovery, ¡2-second matching, and 85% pickup completion in a local prototype environment.**

I. Executive Summary

This System Design Document constitutes Workshop 2 about the Systems Design of the Food Waste Reduction Platform project and represents the direct continuation of the Workshop 1.

Building on the Workshop 1 findings, the present document derives and satisfies seventeen traceable, quantifiable design requirements (Appendix A). These requirements drive the high-level architecture (Figure 1), component responsibilities (Table I), interface definitions, and data flows.

The selected technology stack (React Native/Expo frontend, NestJS backend, PostgreSQL+PostGIS, Redis for concurrency and caching, Expo/FCM push notifications) enables: strict geospatial enforcement, (serialized multi-criteria matching, staggered notifications to mitigate demand amplification, automatic timeout/reassignment and user reliability scoring to handle no-shows, real-time KPIs and equity monitoring to prevent disengagement and bias. Ten critical risks and chaotic dynamics identified in Workshop 1 are explicitly mitigated via architectural patterns, monitoring mechanisms, and control loops (Appendix A).

Implementation is organized in three sequential six-week phases aligned with the semester. The resulting prototype will demonstrate end-to-end redistribution, real-time tracking, impact quantification, and satisfaction of principal success thresholds.

II. Requirements Analysis

A. Synthesis of Workshop 1 Findings

The Workshop 1 analysis provided a structured examination of the Food Waste Reduction Platform, defining its system boundaries, operational constraints, stakeholders, and key environmental conditions. The analysis also revealed several performance bottlenecks, coordination risks, and sensitivity factors that may affect system reliability and scalability. The following findings synthesize the most relevant insights extracted from the Workshop 1 deliverable and serve as the basis for deriving measurable design requirements for the platform.

- **Strict 3 km operational radius** (Section I-C and I-B.2): The system is geographically constrained to a 3 km radius around the Engineering Faculty headquarters, directly impacting geolocation accuracy and matching logic.
- **Same-day publication and collection windows** (Section I-D): Surplus can only be published 2–4 hours before closing with a maximum 1–3 hour collection window to guarantee freshness and food safety.
- **Mandatory priority for verified charities** (Section I-F.2 and I-G.1): Large-quantity offers must prioritize charities/foundations, establishing a clear social equity requirement.
- **Geolocation as a critical and fragile input** (Section III-C.1): Accurate GPS data is essential for all matching; service failures represent a single point of failure.
- **No physical transport, quality verification or payment within system boundaries** (Section I-B.2): The platform is limited to digital coordination and tracking only.
- **High sensitivity to no-shows and confirmed-but-uncollected offers** (Section III-C.1): A single no-show can trigger cascading reassignments and increase food waste risk.
- **Risk of informal external coordination networks** (Section III-C.2): Users may bypass the algorithm via WhatsApp groups, distorting fair matching.

- **Automatic impact metrics as core output** (Section I-F.3 and III-E): The system must automatically calculate and expose KPIs such as kg recovered and success rate.
- **High concurrency during peak hours** (Section III-C.1): Demand spikes at lunch and dinner create scalability challenges.
- **Risk of supplier disengagement if offers remain unclaimed** (Section III-C.2): Repeated unclaimed surpluses may cause cafeterias and restaurants to stop participating.
- **Potential inequitable distribution due to proximity bias** (Section III-C.2): The matching algorithm may systematically disadvantage users at the edge of the 3 km radius.
- **Real-time notification responsiveness requirement** (Section III-C.1 and III-D): The system relies on immediate user awareness of newly published surpluses to ensure rapid claims within the limited freshness window. Delays in notification delivery may reduce the probability of successful redistribution.

B. Measurable Design Requirements

Table IV (see Appendix A) presents the complete set of 17 measurable design requirements derived from the Workshop 1 findings. Each requirement is traceable to a specific finding, includes a quantified target, and is classified by category (Functional Constraint, Reliability, Performance, Scalability, Security, Availability, Usability, or Maintainability).

III. System Architecture

A. High-Level Architecture Diagram

Figure 1 (see also Appendix 2) presents the high-level architecture of the food surplus redistribution platform, illustrating the main system components, their interactions, and the data flow between modules.

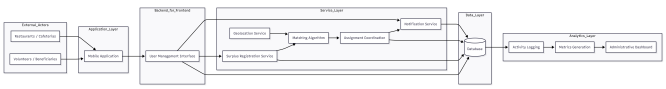

Fig. 1. High-level architecture of the food surplus redistribution platform showing the main components, subsystems, and data flows.

B. Component Responsibilities

TABLE I
Component Responsibilities

Component	Type	Responsibility
Mobile App	UI	Allows users to register, publish food surplus, receive notifications and track food collection.
User Mgmt	Service	Handles user authentication, profile management and role management (donors, volunteers, beneficiaries).
Surplus Module	Service	Registers food surplus data such as type, quantity, expiration time and availability.
Geo Service	Service	Manages location data and calculates proximity between donors and potential collectors.
Matching Engine	Logic	Matches available surplus with nearby volunteers or beneficiaries using location and availability data.
Assignment Module	Service	Assigns collection tasks and coordinates pickup logistics between donors and collectors.
Tracking Module	Service	Monitors the status of food collection and confirms delivery completion.
Notification Service	Service	Sends alerts and updates regarding assignments, confirmations and system events.
Database	Data	Stores users, surplus records, assignments, system logs and operational data.
Analytics Module	Analytics	Processes system activity data to generate metrics and performance indicators.
Admin Dashboard	UI	Provides administrators with monitoring tools, reports and system statistics.

C. Interfaces and Data Flows

The User Interface flow diagrams and System Data Flow diagrams are located in the repository folder Workshop$_2/design/diagrams/interface-and-data-flows$. These diagrams illustrate, in a detailed and sequential manner, the complete flow of information within the Food Waste Reduction Platform: from the publication of a surplus by the supplier, through the matching algorithm, notification delivery, the claimant's acceptance or rejection, and finally to the confirmation of collection and the update of impact metrics.

TABLE II
System Interfaces and Process Data Flows

Interface	Source → Destination	Data Exchanged	Protocol
User Management API	Mobile App → User Service	Credentials (auth), profile details, and role assignment (Donor/Recipient).	HTTPS
Surplus Registration	Mobile App → Surplus Service	Food item details (type, quantity, expiry timestamp) and image URLs.	HTTPS
Geolocation API	Mobile App → Geo Service	Real-time GPS coordinates (latitude, longitude) for 3km radius filtering.	HTTPS

IV. Implementation Strategy

A. Technology Stack

The technology stack was selected based on three criteria: direct alignment with the system's real-time coordination requirements, team familiarity with the frameworks, and suitability for an academic prototype with constrained resources.

1) Frontend — React Native + Expo: React Native with Expo is adopted as the mobile framework for three primary reasons. First, it enables a single codebase for both iOS and Android, reducing development overhead for an academic team. Second, Expo Go facilitates on-device testing without

Interface	Source → Destination	Data Exchanged	Protocol
Matching Engine	Geo Service → Matching Algorithm	Active user IDs, proximity scores, and food urgency priority levels.	Internal
Notification API	Notification Service → Mobile App	Push payload containing assignment alerts, surplus details, and response timeout.	Push
Closure & Verification	Mobile App → Assignment Service	QR code verification data or photo evidence to confirm successful collection.	HTTPS

TABLE III
EXTERNAL SERVICES AND REQUIREMENT COVERAGE

Service	Role	REQ
Google Maps API (fallback: OpenStreetMap)	Geolocation + 3 km radius filtering	REQ-01, REQ-04
Expo Push + FCM	Staggered push notifications	REQ-12
Cloudinary	Surplus image storage	—
Docker Compose	Local orchestration of all services	REQ-15
Sentry + Grafana	Error tracking + infrastructure metrics	REQ-08

a native build step, accelerating iteration cycles. Third, the Expo ecosystem provides native access to GPS, camera (for photo-based collection confirmation), and push notifications via Expo Notifications and FCM — all components critical to REQ-04, REQ-12, and REQ-14. State management is handled by **Zustand** (lightweight, no boilerplate) and **React Query** for data fetching with client-side cache, reducing redundant API calls during concurrency peaks (REQ-09).

2) Backend — NestJS (Node.js): NestJS is selected as the backend framework given the team's existing proficiency with it. Its decorator-based modular architecture facilitates the separation of SurplusModule, MatchingModule, NotificationModule, and AnalyticsModule with low coupling, directly mirroring the four-subsystem decomposition established in Workshop 1. NestJS provides native WebSocket support via @nestjs/websockets for real-time collection tracking (REQ-14) and integrates with **Bull** queues over Redis to implement a serialized matching engine that prevents race conditions (REQ-09). Prisma ORM is preferred over TypeORM for its compile-time type safety and more predictable migration workflow.

3) Database — PostgreSQL + Redis: PostgreSQL serves as the primary persistence layer due to its native support for geospatial types via the **PostGIS** extension, enabling the 3 km radius constraint (REQ-01) to be enforced directly in SQL using ST_DWithin queries. Redis fulfills a dual role: caching frequent matching results and storing JWT session tokens, reducing load on PostgreSQL during peak concurrency windows at lunch and dinner hours (REQ-09).

4) External Services: Table III summarizes the external services integrated into the platform, their functional roles, and the requirements they address.

B. Implementation Phases

The implementation is structured in three sequential phases totaling 6 weeks, aligned with the 2026-I academic semester. Each phase maps directly to the measurable design requirements defined in Table IV.

1) Phase 1 — Core Infrastructure (Weeks 1–2): Objective: Deliver a navigable MVP covering the basic surplus publication and listing flow.

- Authentication module: registration and login with JWT, roles `DONOR | RECIPIENT | CHARITY | ADMIN`.
- Surplus CRUD with temporal window validation (REQ-02) and 3 km radius restriction via PostGIS (REQ-01).
- Mobile screens: available surplus listing, donor publication form, pickup point map.
- Local environment setup via Docker Compose, orchestrating PostgreSQL, Redis, and NestJS with hot-reload.
- CI/CD pipeline with GitHub Actions: lint and unit tests on every pull request.

Deliverable: Functional app navigable in Expo Go, testable by the team on a shared local network.

2) Phase 2 — Matching & Coordination (Weeks 3–4): Objective: Implement the redistribution core with all critical business rules.

- Matching engine: proximity-based algorithm weighted as 50% distance + 30% fairness score + 20% charity priority (REQ-03, REQ-11), implemented as a Bull queue job to enforce request serialization and prevent race conditions (REQ-09).
- Staggered push notifications by proximity tier: first 500 m, then 1 km, then 3 km, with a maximum flow of 50 simultaneous notifications per surplus (Table **??**, demand amplification mitigation).
- Automatic 15-minute timeout and reassignment with compressed time window (REQ-06); maximum of 3 reassignments per surplus item.
- Dual collection confirmation: QR scan or photo upload from the mobile app (REQ-14).
- WebSocket-based real-time tracking of collection status.
- Redis session throttling and anomalous acceptance pattern detection (REQ-07).
- Geolocation fallback: manual address input with server-side radius validation (REQ-04).

Deliverable: End-to-end redistribution system running locally, including all error flows (no-show, rejection, expiration).

3) Phase 3 — Analytics & Prototype Validation (Weeks 5–6): Objective: Deliver a locally validated prototype demonstrating the full platform behaviour.

- Administration dashboard with real-time KPIs: kilograms recovered, redistribution success rate, average matching time, and supplier activity trend (REQ-08, Table VI).
- Equity index: report of geographic assignment distribution to detect proximity bias (REQ-11).

- Automated alerts: `unclaimed_rate > 20%` over 3 consecutive days per supplier (REQ-10); automatic 48-hour account restriction after 3 no-shows within 7 days (REQ-17).
- Sentry integration for local error tracking; Grafana for infrastructure metrics visualization on the local Docker stack.
- User reliability score: initialized at 100, decremented by 20 points per no-show, with restriction triggered at the defined threshold (REQ-17).
- Functional demonstration with simulated donor and recipient accounts on the local environment.

Deliverable: Fully functional local prototype, Swagger API documentation, and functional validation report.

C. Deployment Strategy and Interface Standards

1) Local Deployment Environment: All system components are orchestrated locally via Docker Compose throughout the six-week development period. The stack comprises three containers: a PostgreSQL instance with the PostGIS extension enabled, a Redis instance for session management and queue processing, and the NestJS API server with hot-reload enabled for development. The mobile application runs on physical or emulated devices via Expo Go, connecting to the local API over the shared development network.

No remote or cloud deployment is planned within the scope of this prototype. All validation and functional demonstrations are performed on the local environment, which is considered sufficient to verify the measurable targets defined in Table IV.

2) Interface Standards: All REST interfaces follow the protocol and data exchange specifications defined in Table **??**: HTTPS for synchronous endpoints, WebSockets for tracking events, and JSON payloads with ISO 8601 timestamps. Critical endpoints for matching and notification delivery include idempotency keys to prevent duplicate processing under retry conditions. The API is self-documented using NestJS ApiProperty decorators with Swagger UI, available at /api/docs on the local development server.

V. Risk and Complexity Management

By recognizing the sensitivity factors, emergent behaviors, and chaotic dynamics described in Workshop 1, these can be classified as follows:

1) Critical Input Sensitivities:

- **High-Demand Surplus Events**: Many users attempt to claim the same surplus at the same time, creating a risk of race conditions and allocation conflicts.
- **Large Volumes of Simultaneous Users**: Peak concurrency during busy hours may cause response time degradation and possible database inconsistencies.
- **Geolocation Service Failure**: If there is an external failure of GPS or the maps API, the matching algorithm stops because it cannot calculate distances.
- **Confirmed Match with No Pickup**: The user accepts but does not collect within the defined time, causing the surplus to remain locked and eventually be lost.

2) Emergent Behaviors:

- **Informal Priority Networks Among Users**: There is a possibility that WhatsApp groups or external agreements may emerge, bypassing the algorithm and distorting fairness.
- **Supplier Disengagement Under Low Demand**: If surpluses are not claimed, suppliers may stop offering them, creating a negative cycle for the system.
- **Inequitable Distribution Patterns**: The algorithm consistently favors the closest users, which may lead peripheral users to disengage.

3) Chaotic System Dynamics:

- **Nonlinear Cascade from a Single No-Show**: One no-show triggers successive reassignments that consume time until the surplus expires.
- **Demand Amplification Through Notifications**: Simultaneous notifications to many users may create an artificial demand spike that collapses the system.
- **Feedback Loop Between Metrics and Supplier Behavior**: If the dashboard shows low performance rates, suppliers may become demotivated and post fewer surpluses, further worsening the metrics.

Based on these ten factors, the following risk and complexity table has been developed (see Table V in Appendix A), designing specific mitigation mechanisms for each factor, as well as proposing monitoring, feedback, and control strategies.

VI. Performance and Quality Assurance

To ensure that the Food Waste Reduction Platform operates efficiently, reliably and at scale, the system incorporates a structured performance monitoring framework based on measurable Key Performance Indicators (KPIs), continuous monitoring strategies, and a feedback-driven improvement cycle.

A. Key Performance Indicators

The following KPIs are used to evaluate system effectiveness, operational performance and social impact (see Table VI in Appendix A).

B. Monitoring Strategy

The platform includes an operational monitoring layer designed to continuously track system performance and detect anomalies in real time.

The monitoring strategy includes the following mechanisms:

- **Operational Logging**: All system events (surplus publication, matching operations, notifications, and pickups) are recorded in centralized logs.
- **Real-Time Metrics Collection**: Key system metrics such as response time, concurrent users, and assignment success rate are collected continuously by the analytics module.
- **Administrative Dashboard**: The Admin Dashboard provides real-time visualization of operational indicators such as recovered kilograms, success rate, supplier activity trends, and pickup completion rates.

- **Automated Alerts**: Alerts are triggered when predefined thresholds are exceeded, such as high no-show rates, increasing response times, or decreasing surplus recovery rates.

These monitoring mechanisms allow administrators to identify performance issues quickly and respond before they significantly impact the system.

C. Success Criteria and Continuous Improvement Plan

System success is evaluated based on measurable operational outcomes and long-term participation sustainability.

The platform is considered to be operating successfully when the following criteria are consistently met:

- At least **80% of published surplus food is successfully redistributed**.
- The **matching algorithm generates assignments within 2 seconds**.
- The **pickup completion rate remains above 85%**.
- Push notifications are delivered in **less than 5 seconds for at least 95% of users**.

To ensure long-term improvement, the platform follows a continuous improvement cycle:

- **Weekly metric reviews** performed by system administrators to detect operational trends.
- **Algorithm tuning** based on observed matching fairness, pickup reliability, and geographic distribution.
- **User behavior analysis** to detect repeated no-shows or anomalous claiming patterns.
- **System performance optimization** based on observed load patterns during peak hours.

This iterative monitoring and improvement approach ensures that the platform remains scalable, fair, and effective in reducing food waste while maintaining high operational reliability.

VII. Conclusion and Next Steps

This Workshop 2 Systems Design Document has successfully translated the rigorous analysis, system boundaries, temporal and geospatial constraints, input-process-output model, and risk and sensitivity factors identified in Workshop 1 into a coherent, traceable, and implementable technical architecture for the Food Waste Reduction Platform.

The proposed design directly satisfies the 17 measurable design requirements (Appendix A) while explicitly mitigating the 10 critical risks, emergent behaviors, and chaotic dynamics documented in Workshop 1 (Appendix A). Key architectural decisions ensure strict enforcement of the 3 km radius and same-day freshness windows, prevent race conditions during peak concurrency, promote equitable redistribution, sustain supplier engagement, and generate verifiable KPIs.

By adopting a phased, locally orchestrated implementation strategy, the project maintains reproducibility, facilitates peer review, and aligns with best practices in systems engineering and computer science collaboration. The resulting prototype foundation enables systematic testing, complexity analysis, performance benchmarking, and user validation in subsequent phases, demonstrating the platform's potential to meaningfully reduce food waste in Bogotá context while advancing circular economy principles and social equity.

Next steps include:

- Proceed to **Workshop 3 — Project Management and Implementation**: Complete the three-phase development plan, conduct unit/integration testing, execute simulation scenarios with realistic data (e.g., peak-hour concurrency, no-show cascades), measure against defined KPIs and requirements, perform risk reassessment, and document quality assurance results including complexity analysis and performance benchmarks.
- Advance to **Workshop 4 — Final Results and Project Communication**: Finalize the working prototype with end-to-end user validation (simulated and controlled real-user testing), analyze experimental outcomes, refine the matching algorithm based on observed equity and recovery metrics, and prepare the full set of academic deliverables.

References

[1] "Food Waste Reduction Mobile Application, A Rigorous System Analysis," Workshop 1 Deliverable, 2026.

Appendix

The IV table presents the complete set of measurable design requirements derived from the Workshop 1 findings. Each requirement is traceable to a specific finding, includes a quantified target, and is classified by category.

Figure 2 presents the full-resolution architecture diagram of the Food Waste Reduction Platform. The diagram illustrates the interaction between the mobile application, service modules, matching engine, database layer, and analytics subsystem.

Table V presents the complete analysis of system risks and complexity, developed from the 10 sensitivity factors, emergent behaviors, and chaotic dynamics identified in Workshop 1.

Table VI shows the KPIs used to evaluate system effectiveness.

TABLE IV
MEASURABLE DESIGN REQUIREMENTS DERIVED FROM WORKSHOP 1

Req ID	Workshop 1 Finding	Design Requirement	Measurable Target	Category
REQ-01	Operational radius limited to 3 km from Engineering Faculty	Geolocation module shall restrict all offers and matches to users within the defined radius	Maximum distance 3 km with $\pm$50 m accuracy (100% enforcement)	Functional Constraint
REQ-02	Same-day publication and collection window	System shall enforce same-day publication and strict collection window	Publication: 2–4 hours before closing; collection $\leq$ 3 hours (100% compliance)	Reliability
REQ-03	Mandatory priority for verified charities on large quantities	Matching algorithm shall apply priority weighting to verified charity accounts	$\geq$90% of offers $\geq$10 kg assigned to charities first	Reliability
REQ-04	Geolocation as critical input and single point of failure	System shall validate geolocation in real time with fallback mechanism	Automatic rejection if invalid; fallback available in $<$5 seconds	Reliability
REQ-05	Transport, quality verification and payment outside system boundaries (Section I-B.2)	Platform shall restrict its functionality exclusively to digital coordination, reservation and collection tracking of food surplus	0 modules related to payment processing, physical transport management or food quality verification present in the system architecture (verified by code review and system testing)	Maintainability
REQ-06	High sensitivity to no-shows	System shall implement automatic timeout and re-assignment	Timeout = 15 minutes; re-assignment success rate $\geq$85%	Reliability
REQ-07	Risk of informal external coordination networks (Section III-C.2)	System shall monitor anomalous acceptance patterns	Detection and logging of $>$3 coordinated acceptances from the same user group within a 30-minute window for the same offer (audit log mandatory)	Security
REQ-08	Impact metrics must be calculated automatically	System shall aggregate and update KPIs in real time	Daily dashboard showing kg recovered and success rate (updated $<$60 s after each collection)	Performance
REQ-09	High concurrency during peak hours	Architecture shall support concurrent users without degradation	Handle $\geq$200 concurrent users with matching time $\leq$30 s	Scalability
REQ-10	Risk of supplier disengagement if offers remain unclaimed	System shall generate early-warning alerts for administrators	Alert triggered when unclaimed rate $>$20% for any supplier during 3 consecutive days	Reliability
REQ-11	Proximity bias causing inequitable distribution	Matching algorithm shall include distance-balancing mechanism	Peripheral users (2.5–3 km) receive $\geq$15% of total matches	Reliability
REQ-12	Real-time notification responsiveness requirement	System shall deliver surplus notifications to candidate users in real time	Push notification delivery $<$5 seconds for $\geq$95% of users	Performance
REQ-13	Matching algorithm processing time constraint	Matching algorithm shall generate a candidate assignment within bounded execution time	Matching decision generated in $<$2 seconds for $\geq$95% of surplus publications	Performance
REQ-14	Need for reliable pickup confirmation process	System shall require dual confirmation of surplus collection from supplier and recipient	100% of completed redistributions require supplier confirmation and recipient confirmation	Reliability
REQ-15	Dependence on cloud infrastructure availability	Platform infrastructure shall maintain high service availability	System uptime $\geq$99% measured monthly	Availability
REQ-16	Mobile application responsiveness requirement	Mobile interface shall load surplus listings quickly for users	Listing screen load time $<$2 seconds for $\geq$95% of sessions	Usability
REQ-17	Repeated no-show user behavior risk	System shall track user reliability and restrict repeated no-show behavior	Users with $>$3 no-shows within 7 days receive temporary restriction of 48 hours	Reliability

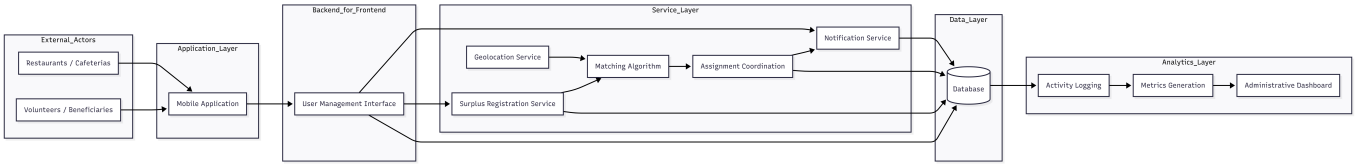

Fig. 2. High-Level Architecture of the Food Surplus Redistribution Platform

TABLE V
COMPLEXITY AND RISK MITIGATION

Risk (Workshop 1)	**Category**	**Risk Level**	**Specific Mitigation Mechanism**	**Monitoring and Feedback Strategies**	**Error Handling Mechanisms**	**Failure Mode and Resilience**
High-Demand Surplus Events	Critical Input	High	Serialization with DB locking (`SELECT FOR UPDATE`) + message queue. Limit of 1 claim per surplus.	Real-time `contention_rate` metric (admin dashboard). Email/Slack alert if $> 20\%$.	60-second acceptance timeout. "Fair lottery" mode if > 5 simultaneous requests.	Concurrency failure $\rightarrow$ system remains operational (only the first claim succeeds).
Large Volumes of Simultaneous Users	Critical Input	High	Automatic backend scaling. Rate limiting per IP/user.	`concurrent_users` and average response time metrics (Prometheus + Grafana).	Read caching (Redis) + fallback to offline mode (local data in the app).	Load spike $\rightarrow$ controlled degradation ("loading…" response instead of crash).
Geolocation Service Failure	Critical Input	High	Manual fallback + last-known-location cache. Use of OpenStreetMap as backup for Google Maps.	`geo_failures_per_hour` metric with automatic alerts.	Manual address input + validation within a 3 km radius.	Without geolocation $\rightarrow$ matching is performed using manual radius (partial resilience).
Confirmed Match with No Pickup	Critical Input	High	User reliability score (starts at 100, decreases by 20 points per no-show). Automatic reassignment after timeout.	Dashboard: `no_show_rate` per user and globally. Push notification to supplier.	Dynamic timeout (30–90 minutes depending on remaining expiration time).	Surplus is automatically released $\rightarrow$ it never remains locked.
Informal Priority Networks Among Users	Emergent Behavior	Medium	Detection of anomalous patterns (repeated acceptances in < 10 s). "External coordination suspected" flag.	Weekly "acceptance anomalies" report in the dashboard.	Temporary ranking penalty in the algorithm if a pattern is detected.	Fairness maintained even if external groups exist.
Supplier Disengagement Under Low Demand	Emergent Behavior	Medium	Automatic recommendation system and push reminders for suppliers.	`supplier_activity_trend` metric (posts/week). Alerts if it drops $> 30\%$.	Personalized supplier dashboard showing "real impact" (kg recovered).	Positive feedback prevents the abandonment cycle.
Inequitable Distribution Patterns	Emergent Behavior	Medium	Hybrid algorithm: 50% distance + 30% fairness score (peripheral users move up in ranking).	`equity_index` metric (geographic distribution of claims).	Dynamic weight adjustment based on weekly analysis.	Distant users periodically receive priority $\rightarrow$ prevents peripheral disengagement.
Nonlinear Cascade from a Single No-Show	Chaotic Dynamics	High	Chain reassignment with compressed time window (reduced by 50% at each reassignment).	Cascade visualization in dashboard + "active chain" alert.	Maximum limit of 3 reassignments per surplus.	Chain automatically stops before total expiration.
Demand Amplification Through Notifications	Chaotic Dynamics	High	Staggered notifications by proximity tier (first 500 m, then 1 km, then 3 km).	`notification_spike` metric (claims/minute).	Flow control (max 50 simultaneous notifications per surplus).	Artificial demand spikes are distributed over time $\rightarrow$ prevents system collapse.
Feedback Loop Between Metrics and Supplier Behavior	Chaotic Dynamics	Medium	Contextualized presentation of metrics (trends + Bogotá benchmark). Individualized report sent to suppliers.	Automatic alert if `publication_rate` drops for 2 consecutive weeks.	Dashboard with motivational messages ("You saved 42 kg this week!").	Breaks the negative cycle by highlighting real impact.

TABLE VI
KEY PERFORMANCE INDICATORS

KPI	Description	Target Value
Surplus Recovery Rate	Percentage of published food surplus that is successfully collected.	$\geq$80%
Average Matching Time	Time required by the matching engine to assign a collector after publication.	$<$2 seconds
Successful Pickup Rate	Percentage of confirmed assignments that result in completed pickups.	$\geq$85%
Notification Delivery Time	Time required to deliver push notifications to candidate users.	$<$5 seconds
System Response Time	Average backend API response time during normal operation.	$<$500 ms
Concurrent User Capacity	Maximum number of simultaneous active users supported without performance degradation.	$\geq$200 users